

I need you to analyze a markdown file with coding rules for me, and then address the contradictions and ambiguities in them one-by-one. At the end I will need you to add to or update the rules accordingly, and generate a new markdown file with coding rules

claude-opus-4-6-v1

Thought for less than a second ▾

I'd be happy to help analyze your coding rules markdown file for contradictions and ambiguities! However, it looks like you haven't included the file yet.

Could you please share the markdown file contents? You can either:

1. **Paste the contents** directly into your next message
2. **Upload the file** as an attachment

Once I have the rules, I'll:

1. **List each contradiction** I find (where two or more rules conflict with each other)
2. **List each ambiguity** (where a rule is unclear, vague, or open to multiple interpretations)
3. **Propose a resolution** for each issue
4. **Generate a clean, updated markdown file** with the revised rules

Go ahead and share the file whenever you're ready!

here are the contents of the markdown file

Consolidated Naming Convention Rules

Below is the unified rule set, followed by a dedicated section on contradictions and ambiguities.

1. Class Names (Models, Blocks, Packages)

1.1 Class names shall start with an uppercase letter and be a noun, or a combination of adjectives and nouns.

1.2 Use camelCase to combine multiple words (e.g., `HeatTransfer`).

1.3 Do not repeat higher-level package names. Use `Chillers.Carnot` , not `Chillers.CarnotChiller` . Similarly, avoid descriptors in names that are already obvious from the enclosing package (e.g., avoid `boi` for boiler if the block is already in a `BoilerPlant` package).

> Sources: Set 1 Rule 1, Set 3 Rule 24

2. Instance and Variable Names

2.1 General Structure

2.1.1 Instance names (variables, block instances, parameters) shall generally start with a lowercase letter, unless they represent a physical quantity conventionally capitalized (e.g., `T` for temperature, `P` for power, `x` for mass fraction).

2.1.2 Names are composed following this pattern:

Position	Element	Examples
----------	---------	----------

---	---	---
-----	-----	-----

1. Prefix (<i>optional</i>)	<code>u</code> for input, <code>y</code> for output — include only if needed for context	<code>uOpeMod</code> , <code>yHeaCoi</code>
-------------------------------	--	---

2. Quantity (<i>required</i>)	The physical quantity or descriptor	<code>TOut</code> , <code>TZonHea</code> , <code>dpBui</code>
---------------------------------	-------------------------------------	---

3. Postfix/Qualifier (<i>optional</i>)	Modifiers such as setpoint, min, max, cooling, heating, etc., appended after the quantity	<code>TZonHeaSet</code> , <code>TSupSetMin</code>
--	---	---

2.1.3 Strive for short names. Omit prefix or postfix when the meaning is clear from context. For example, a room thermostat may simply use `T` as its input.

> Sources: Set 1 Rules 2 & 5, Set 2 general conventions, Set 3 Rule 6

2.2 Abbreviation Conventions

2.2.1 Use abbreviations composed of approximately the first three characters of each word, combined in camelCase (e.g., `preDro` for pressure drop, `worTim` for working timer, `zonDisEffHea` for zone distribution effectiveness heating).

2.2.2 Single characters may be used where generally understood (e.g., `T`, `p`, `k`, `n`, `u`, `y`). These may be augmented with descriptors as needed (e.g., `yVal` for valve signal, `yDam` for damper signal, `kCoo` for cooling gain).

> Sources: Set 1 Rule 2, Set 2 examples

2.3 Cross-Package Consistency

2.3.1 If an analogous sequence exists for another system (e.g., heating vs. cooling, chiller plant vs. boiler plant), use **consistent, parallel names** across both to facilitate commonization.

2.3.2 For feedback delay `pre` block instances, use **numbered names** `pre1` , `pre2` , etc.) rather than bare `pre` .

> *Sources: Set 3 Rules 25 & 26*

3. Standard Physical Quantity Variables

3.1 The following single-character or short variables are standard:

Variable	Meaning
---	---
<code>T</code>	Temperature
<code>p</code>	Pressure
<code>dp</code>	Pressure difference
<code>P</code>	Power
<code>E</code> (or <code>q</code>)	Energy (or thermal energy)
<code>x</code>	Mass fraction per total mass
<code>x</code>	Mass fraction per mass of dry air
<code>Q_flow</code>	Heat flow rate
<code>m_flow</code>	Mass flow rate
<code>H_flow</code>	Enthalpy flow rate
<code>V_flow</code>	Volume flow rate
<code>k</code>	Gain (e.g., <code>kCoo</code>)
<code>Ti</code>	Integrator time constant (e.g., <code>TiCoo</code>)
<code>Td</code>	Derivative time constant (e.g., <code>TdCoo</code>)
<code>n</code>	Count / number (e.g., <code>nChi</code> for number of chillers)
<code>A</code>	Area (e.g., <code>AFlo</code> for floor area)
<code>h</code>	Specific enthalpy (e.g., <code>hOut</code> for outdoor air enthalpy)

> *Sources: Set 1 Rules 3–4, Set 2 tables*

4. Control Signal Naming

4.1 Control input signals start with `u` ; control output signals start with `y` .

4.2 The `uy` prefix may be omitted when the physical quantity already makes the direction clear.

4.3 When a controller has multiple outputs or inputs, augment with a descriptor (e.g., `yVal` and `yDam` for valve and damper output signals; `uHea` and `uCoo` for heating and cooling input signals).

> Sources: Set 1 Rules 2 & 5, Set 2 conventions

5. Prefix Conventions

5.1 The following prefixes are standard:

Prefix	Meaning	Examples
---	---	---
<code>u</code>	Control input signal	<code>uOpeMod</code> , <code>uDam</code> , <code>uHea</code> , <code>uCoo</code>
<code>y</code>	Control output signal	<code>yHeaCoi</code> , <code>yCooCoi</code> , <code>yPosMin</code> , <code>yChiPumSpe</code>
<code>use_</code>	Conditionally enabled input or feature (boolean parameter)	<code>use_T_in</code> , <code>use_TMix</code> , <code>use_enthalpy</code>
<code>have_</code>	Indicates the system/zone has a particular component, sensor, or configuration (boolean parameter)	<code>have_CO2Sen</code> , <code>have_occSen</code> , <code>have_winSen</code> , <code>have_parChi</code>
<code>need_</code>	Indicates a required capability or constraint (boolean parameter)	<code>need_reduceChillerDemand</code>

> Sources: Set 1 Rule 6, Set 2 tables

6. Suffix / Postfix Conventions

6.1 The following suffixes are standard:

Suffix	Meaning	Examples
---	---	---
<code>_flow</code>	Flow variable	<code>Q_flow</code> , <code>m_flow</code> , <code>V_flow</code> , <code>VDis_flow</code>
<code>_nominal</code>	Design / nominal capacity	<code>Q_flow_nominal</code> , <code>m_flow_nominal</code> , <code>V_flow_nominal</code>
<code>_small</code>	Small value used for regularization	—
<code>Set</code>	Setpoint	<code>TZonHeaSet</code> , <code>TZonCooSet</code>

Set	Setpoint	TZoneCoolSet	TZoneHeatSet
Min / Max	Minimum / Maximum	TSupSetMin	
TZonCooMax	VDisHeaSetMax_flow		
_remote	From a remote sensor	dpChiWat_remote	
Hea / Coo	Heating / Cooling qualifier	TZonHeaSet	
yCooCoi	zonDisEffCoo		

6.2 When multiple suffixes are needed, follow this general order: *Quantity* → *Context/Qualifier* → *Setpoint* → *Min/Max* → *_flow_nominal*. For example:

VDisHeaSetMax_flow.

> *Sources: Set 1 Rule 6, Set 2 tables and examples*

7. Connector Naming

7.1 Paired connectors of the same domain (identical declarations, different icons) are distinguished by:

Convention	Usage	Example
---	---	---
_a / _b	Fluid ports	port_a , port_b
_p / _n	Electrical terminals	terminal_p , terminal_n

> *Source: Set 1 Rule 7*

8. Psychrometrics Package Naming

Buildings.Utilities.Psychrometrics)

8.1 Uppercase *x* = mass fraction per total mass; lowercase *x* = mass fraction per mass of dry air.

8.2 The notation *z_xy* denotes a function or block with output *z* and inputs *x* and *y*.

8.3 Standard symbols:

Symbol	Meaning
---	---
pW	Water vapor pressure
TDewPoi	Dew point temperature
TWetBul	Wet bulb temperature
TDryBul (or T)	Dry bulb temperature

> *Source: Set 1 Rule 4*

Contradictions and Ambiguities

Contradictions and Ambiguities

Contradiction 1: `_min_max` vs. `MinMax`

SET 1 (EXPLICIT RULE)	SET 2 (EXAMPLES)
Suffix <code>_min</code> / <code>_max</code> (underscore + lowercase), e.g., <code>TSupCoo_min</code>	Suffix <code>Min</code> / <code>Max</code> (camelCase, no underscore), e.g., <code>TSupSetMin</code> , <code>TZonCooMax</code> , <code>dpChiWatPumMin</code>
Set 2 states that it follows the conventions of Set 1, yet its examples overwhelmingly use <code>MinMax</code> in camelCase without an underscore. Confusingly, Set 2 also contains <code>lift_min</code> , which follows the Set 1 convention.	
Recommendation: Adopt one format and apply it consistently. The <code>MinMax</code> camelCase style is more consistent with other postfix qualifiers (<code>Set</code> , <code>Hea</code> , <code>Coo</code>) that also have no underscore. If that is chosen, the Set 1 rule and the <code>lift_min</code> example in Set 2 should be updated.	

Contradiction 2: `min` used as a prefix vs. postfix

Set 2 contains the parameter `minChiLif` (minimum chiller lift), where `min` appears as a **prefix**. This directly contradicts:

- Set 1 Rule 6 (suffixes `_min_max`)
- Set 2's own stated convention ("as a postfix a qualifier may be added, such as... `Min` for minimum")
- Set 3 Rule 6 ("modifiers like... minimum, maximum... should be appended **after** the variable name")

Recommendation: Rename `minChiLif` to something like `chiLifMin` or `lift_min` to follow the postfix convention.

Ambiguity 1: `need_` prefix not formally defined

The prefix `need_` appears in Set 2's examples (`need_reduceChillerDemand`) but is never explicitly defined as a convention in either Set 1 or Set 2's descriptive text (unlike `use_` and `have_` , which are). Its intended semantic distinction from `have_` is unclear.

Recommendation: Formally define `need_` and clarify how it differs from `have_` . A possible distinction: `have_` = "the physical system includes this" vs. `need_` = "the control logic requires this behavior."

Ambiguity 2: When to use underscores vs. camelCase in suffixes

Some suffixes use an underscore `_flow` , `_nominal` , `_small` , `_remote`) while others use camelCase with no

underscore `Set`, `Min`, `Max`, `Hea`, `Coo`). No explicit rule governs which style to use.

Possible implicit pattern: Underscored suffixes (`_flow`, `_nominal`, `_small`) tend to be **category markers** that describe the *type* of quantity, while camelCase suffixes (`Set`, `Min`, `Hea`) tend to be **qualifiers** of the specific value. However, `_remote` doesn't fit this pattern neatly.

Recommendation: Explicitly state the rule. For example: *"Use underscore suffixes for quantity-type markers (`_flow`, `_nominal`, `_small`). Use camelCase postfixes for value qualifiers (`Set`, `Min`, `Max`, `Hea`, `Coo`)."*

Ambiguity 3: When to abbreviate vs. use full words

Set 1 Rule 2 says to use "the first three characters of a word," and most examples follow this (`Hea`, `Coo`, `Dis`, `Sup`). However, Set 2 contains full-word names like `samplePeriod`, `lift_min`, and `retDamPhyPosMax`. No rule clarifies when full words are acceptable or preferred.

Recommendation: Add guidance such as: *"Use three-character abbreviations for domain-specific or frequently used terms. Full words may be used for general programming terms or when abbreviation would be ambiguous."*

Ambiguity 4: Ordering of multiple postfix qualifiers

When a name requires multiple qualifiers, the ordering is inconsistent across Set 2's examples:

Name	Apparent order
<code>TSupSetMin</code>	Quantity → Set → Min
<code>pMinSet</code>	Quantity → Min → Set
<code>VDisHeaSetMax_flow</code>	Quantity → Context → Set → Max → <code>_flow</code>

`TSupSetMin` puts `Set` before `Min`, while `pMinSet` puts `Min` before `Set`.

Recommendation: Standardize the order explicitly. A suggested canonical order: **Quantity → Context**

HeaCooDis /...) → Set → MinMax → _flow_nominal .

Ambiguity 5: Set 3 Rule 6 — "block names" interpretation

Set 3 Rule 6 states "*block names and variables usually start with a lowercase letter.*"

If "block names" refers to **class names**, this contradicts Set 1 Rule 1 (class names start uppercase). If it refers to **instance names** of blocks, it is consistent. Per the user's instructions, if it is a contradiction with Set 1, it is ignored. The consolidated rules above interpret it as referring to instance names.

claude-opus-4-6-v1

Thought for 3 minutes ✓

Analysis of Naming Convention Rules

I've carefully reviewed the full document — both the consolidated rules and the identified contradictions/ambiguities. Below I address each existing issue, identify several new issues, and then provide a fully revised markdown file at the end.

Addressing the Existing Contradictions & Ambiguities

Contradiction 1: _min / _max vs. Min / Max

The conflict: Set 1 defines underscore-lowercase suffixes (_min , _max), but the overwhelming majority of examples across all sets use camelCase without an underscore (Min , Max). The lone outlier is lift_min .

Resolution: Adopt Min / Max (camelCase, no underscore).

Rationale:

- It is consistent with every other value-qualifier suffix (Set , Hea , Coo) — none of which use underscores.
- Min and Max follow the three-character abbreviation rule (first three characters of "Minimum" / "Maximum").
- The underscore-style suffixes (_flow , _nominal , _small) serve a *different* function (see Ambiguity 2 resolution below), and Min / Max don't belong in that category.
- The lift_min example should be corrected to lifMin .

Changes to rules:

- Remove _min / _max from any suffix table.
- Ensure Min / Max are listed as standard value-qualifier suffixes.

Contradiction 2: min used as a prefix (minChiLif)

The conflict: minChiLif puts the modifier *before* the quantity, contradicting the explicit postfix rule in multiple sources.

Resolution: Rename to `chilifMin` .

Rationale: Rules 2.1.2, 6.1, and the sources all agree that modifiers like minimum/maximum are appended *after* the quantity. No exceptions should exist.

Changes to rules:

- Add an explicit note in the suffix section that `Min` , `Max` , `Set` , etc. must always be **postfixes**, never prefixes.

Ambiguity 1: `need_` prefix not formally defined

The conflict: `need_` appears in examples but has no definition, making its distinction from `have_` and `use_` unclear.

Resolution: Formally define `need_` in the prefix table.

Proposed definition:

PREFIX	MEANING
<code>use_</code>	A conditionally enabled input or feature (boolean parameter). Answers: <i>"Should the model use this optional input/feature?"</i>
<code>have_</code>	The physical system has a particular component, sensor, or configuration (boolean parameter). Answers: <i>"Does this system physically include this?"</i>
<code>need_</code>	A required control capability or operational constraint (boolean parameter). Answers: <i>"Does the control logic need to enforce this behavior?"</i>

Changes to rules:

- Add `need_` with its definition and a corrected example to the prefix table.
- Correct the example `need_reduceChillerDemand` → `need_redChiDem` (to also follow abbreviation conventions; see New Issue 4 below).

Ambiguity 2: When to use underscores vs. camelCase in suffixes

The conflict: Some suffixes use underscores (`_flow` , `_nominal` , `_small` , `_remote`) while others use camelCase (`Set` , `Min` , `Max` , `Hea` , `Coo`). No rule explains which style to use.

Resolution: Define two explicit suffix categories.

CATEGORY	FORMATTING	PURPOSE	EXAMPLES
Quantity-type markers	Underscore + full (unabbreviated) word	Classify the variable's fundamental nature or modeling role. These are library-wide or Modelica-wide conventions.	<code>_flow</code> , <code>_nominal</code> , <code>_small</code> , <code>_remote</code>
Value qualifiers	CamelCase, no underscore, three-character abbreviations	Narrow the engineering meaning of the specific variable.	<code>Set</code> , <code>Min</code> , <code>Max</code> , <code>Hea</code> , <code>Coo</code>

The distinguishing principle: **underscore suffixes are full, unabbreviated words; camelCase suffixes follow the three-character abbreviation convention.** This cleanly explains every existing example:

- `_flow` (full word, category marker) vs. `Set` (abbreviation of "Setpoint", value qualifier)
- `nominal` (full word) vs. `Min` (abbreviation of "Minimum")

- `_remote` (full word, describes the measurement source category)

Changes to rules:

- Add a new sub-section in §6 that explicitly defines the two suffix types and their formatting rules.
- Note that new underscore suffixes should only be introduced for broadly applicable concepts used consistently across the library.

Ambiguity 3: When to abbreviate vs. use full words

The conflict: Rule 2.2.1 says to use three-character abbreviations, but examples like `samplePeriod`, `lift_min`, and `use_enthalpy` use full words without explanation.

Resolution: Add explicit guidance on when full words are acceptable.

Full (unabbreviated) words may be used for:

1. General programming terms not specific to the HVAC/building domain (e.g., `samplePeriod`).
2. Standard Modelica/library underscore suffixes (e.g., `_nominal`, `_flow`, `_remote` — see §6).
3. Cases where the three-character abbreviation would be ambiguous or unrecognizable to a domain practitioner.

In all other cases — particularly for HVAC/building-domain terms that appear frequently — the three-character abbreviation rule applies.

Changes to rules:

- Add a new sub-rule 2.2.3 codifying when full words are acceptable.

Ambiguity 4: Ordering of multiple postfix qualifiers

The conflict: `TSupSetMin` puts `Set` before `Min`, but `pMinSet` puts `Min` before `Set`. These are contradictory orderings.

Resolution: Standardize the canonical order and correct `pMinSet` → `pSetMin`.

The canonical order:

Quantity → Context/Domain (`Sup`, `Zon`, `Dis`, ...) → Domain Qualifier (`Hea`, `Coo`, ...) → `Set` → `Min` / `Max` → `_flow` → `_nominal`

Verification against examples:

NAME	DECOMPOSITION	CORRECT?
<code>TSupSetMin</code>	T → Sup → Set → Min	✓
<code>TZonHeaSet</code>	T → Zon → Hea → Set	✓
<code>VDisHeaSetMax_flow</code>	V → Dis → Hea → Set → Max → _flow	✓
<code>pMinSet</code>	p → Min → Set	✗ → should be <code>pSetMin</code>

Changes to rules:

- Make the ordering rule explicit with a table of frequently used quantities

Make the ordering rule explicit with a table of correct decompositions.

- Note that `pMinSet` should be corrected to `pSetMin`.

Ambiguity 5: Set 3 Rule 6 — "block names" interpretation

The conflict: Set 3 Rule 6 says "block names... usually start with a lowercase letter." If "block names" means class names, it contradicts Rule 1.1 (uppercase start). If it means instance names, it's consistent.

Resolution: Interpret as instance names; clarify in the consolidated rules.

The consolidated rules already frame §2 as "Instance and Variable Names" and explicitly list "block instances" in Rule 2.1.1. This makes the distinction clear. I'll add a brief clarifying note to remove any residual ambiguity.

Changes to rules:

- Add a parenthetical note to Rule 2.1.1 clarifying that "block instances" refers to instances (not class/type definitions), which are governed by §1.

New Issues Identified

New Issue 1: "camelCase" terminology for class names (Rule 1.2)

The problem: Rule 1.2 says "Use **camelCase** to combine multiple words" for class names, with the example `HeatTransfer`. However, `HeatTransfer` starts with an uppercase letter, which is technically **UpperCamelCase (PascalCase)**, not camelCase (which starts lowercase, e.g., `heatTransfer`). This creates a terminology conflict with Rule 2.1.1, which uses "camelCase" to mean lowerCamelCase for variables.

Resolution: Use precise terminology:

- §1 (Class names): UpperCamelCase (PascalCase)
- §2 (Instance/variable names): lowerCamelCase

Change: Update Rule 1.2 wording to "Use **UpperCamelCase (PascalCase)**..."

New Issue 2: `E` vs. `Q` for energy

The problem: The standard variables table lists `E (or Q)` with meaning "Energy (or thermal energy)." Presenting them as interchangeable alternatives with a parenthetical is imprecise — they actually have distinct meanings.

Resolution: Separate them into distinct rows:

VARIABLE	MEANING
<code>E</code>	Energy (general)
<code>Q</code>	Thermal energy

New Issue 3: "approximately" in Rule 2 2 1

New Issue 3: approximately in Rule 2.2.1

The problem: "approximately the first three characters" is vague. Does "approximately" mean 2–4 characters? Always 3? Is deviation common?

Resolution: Replace "approximately" with "typically," and add explicit guidance that two or four characters are acceptable when three would be ambiguous or when a natural syllable breakpoint makes a different length clearer. Three remains the strong default.

New Issue 4: Abbreviation conventions for use_ / have_ / need_ parameter names

The problem: The names after use_ / have_ / need_ prefixes are inconsistent:

- **Abbreviated** (following §2.2): have_CO2Sen , have_occSen , have_winSen , have_parChi , use_TMix
- **Full words** (not following §2.2): use_enthalpy , need_reduceChillerDemand

No rule specifies whether these prefixed boolean parameters follow the same abbreviation conventions as other variables.

Resolution: State explicitly that names following use_ / have_ / need_ prefixes should apply the same abbreviation conventions as other variables (§2.2). The general exception for full words (New Issue 3 / Rule 2.2.3) still applies — e.g., use_enthalpy may be acceptable if use_ent would be ambiguous, but need_reduceChillerDemand should become need_redChiDem .

Change: Add Rule 5.2 stating this.

Summary of All Resolutions

#	ISSUE	RESOLUTION	KEY RULE CHANGES
C1	<code>_min/_max</code> vs <code>Min/Max</code>	Adopt <code>Min</code> / <code>Max</code> (camelCase)	§6.2 updated
C2	<code>minChiLif</code> (prefix)	Must be postfix: <code>chiLifMin</code>	§6.3 note added
A1	<code>need_</code> undefined	Define in prefix table	§5.1 updated
A2	Underscore vs camelCase suffixes	Two explicit categories defined	§6.1 added
A3	Abbreviation vs full words	Explicit guidance added	§2.2.3 added
A4	Postfix ordering inconsistent	Canonical order standardized	§6.3 expanded
A5	"block names" ambiguous	Clarified as instance names	§2.1.1 note
N1	"camelCase" for class names	PascalCase terminology	§1.2 updated
N2	<code>E</code> vs <code>Q</code> conflated	Separated into distinct rows	§3.1 updated
N3	"approximately" vague	Replaced with "typically" + guidance	§2.2.1 updated
N4	<code>use_</code> / <code>have_</code> / <code>need_</code> abbreviation	Must follow §2.2 conventions	§5.2 added

Updated Markdown File

Consolidated Naming Convention Rules (Revised)

Below is the unified rule set. All previously identified contradictions and ambiguities have been resolved; a change log appears at the end.

1. Class Names (Models, Blocks, Packages)

****1.1**** Class names shall start with an ****uppercase letter**** and be a noun, or a combination of adjectives and nouns.

****1.2**** Use ****UpperCamelCase (PascalCase)**** to combine multiple words (e.g., ``HeatTransfer``, ``BoilerPolynomial``).

****1.3**** Do not repeat higher-level package names. Use ``Chillers.Carnot``, not ``Chillers.CarnotChiller``. Similarly, avoid descriptors in names that are already obvious from the enclosing package (e.g., avoid ``boi`` for boiler if the block is already in a ``BoilerPlant`` package).

> **Sources: Set 1 Rule 1, Set 3 Rule 24**

2. Instance and Variable Names

2.1 General Structure

****2.1.1**** Instance names – including variables, ****block instances**** (i.e., instances of block classes, not the class definitions themselves, which are governed by §1), and parameters – shall start with a ****lowercase letter****, unless they represent a physical quantity conventionally capitalised (e.g., ``T`` for temperature, ``P`` for power, ``X`` for mass fraction).

****2.1.2**** Names are composed following this pattern:

Position	Element	Examples
---	---	---
1. Prefix <i>*(optional)*</i>	<code>`u`</code> for input, <code>`y`</code> for output – include only if needed for context	
2. Quantity <i>*(required)*</i>	The physical quantity or descriptor	<code>`TOut`</code> , <code>`TZonHea`</code> , <code>`dpBui`</code>

| 3. Postfix / Qualifier **(optional)** | Modifiers such as setpoint, min, max, cooling, heating,

****2.1.3**** Strive for ****short names****. Omit prefix or postfix when the meaning is clear from context. For example, a room thermostat may simply use `T` as its input.

> **Sources: Set 1 Rules 2 & 5, Set 2 general conventions, Set 3 Rule 6**

2.2 Abbreviation Conventions

****2.2.1**** Use abbreviations composed of ****the first three characters**** of each word, combined in lowerCamelCase (e.g., `preDro` for pressure drop, `worTim` for working timer, `zonDisEffHea` for zone distribution effectiveness heating). Two or four characters may be used when three characters would be ambiguous or when a natural breakpoint in the word makes a different length clearer.

****2.2.2**** Single characters may be used where generally understood (e.g., `T`, `p`, `k`, `n`, `u`, `y`). These may be augmented with descriptors as needed (e.g., `yVal` for valve signal, `yDam` for damper signal, `kCoo` for cooling gain).

****2.2.3**** ****Full (unabbreviated) words**** may be used for:

- General programming terms not specific to the HVAC/building domain (e.g., `samplePeriod`).
- Standard Modelica / library underscore suffixes (e.g., `\_nominal`, `\_flow`; see §6).
- Cases where the three-character abbreviation would be ambiguous or unrecognisable to a domain practitioner.

In all other cases – particularly for HVAC/building-domain terms that appear frequently – the three-character abbreviation rule applies.

> **Sources: Set 1 Rule 2, Set 2 examples**

2.3 Cross-Package Consistency

****2.3.1**** If an analogous sequence exists for another system (e.g., heating vs. cooling, chiller plant vs. boiler plant), use ****consistent, parallel**

names** across both to facilitate commonisation.

****2.3.2**** For feedback delay ``pre`` block instances, use ****numbered names**** (``pre1``, ``pre2``, etc.) rather than bare ``pre``.

> **Sources: Set 3 Rules 25 & 26**

3. Standard Physical Quantity Variables

****3.1**** The following single-character or short variables are standard:

Variable	Meaning
---	---
<code>`T`</code>	Temperature
<code>`p`</code>	Pressure
<code>`dp`</code>	Pressure difference
<code>`P`</code>	Power
<code>`E`</code>	Energy (general)
<code>`Q`</code>	Thermal energy
<code>`X`</code>	Mass fraction per total mass
<code>`x`</code>	Mass fraction per mass of dry air
<code>`Q_flow`</code>	Heat flow rate
<code>`m_flow`</code>	Mass flow rate
<code>`H_flow`</code>	Enthalpy flow rate
<code>`V_flow`</code>	Volume flow rate
<code>`k`</code>	Gain (e.g., <code>`kCoo`</code>)
<code>`Ti`</code>	Integrator time constant (e.g., <code>`TiCoo`</code>)
<code>`Td`</code>	Derivative time constant (e.g., <code>`TdCoo`</code>)
<code>`n`</code>	Count / number (e.g., <code>`nChi`</code> for number of chillers)
<code>`A`</code>	Area (e.g., <code>`AFlo`</code> for floor area)
<code>`h`</code>	Specific enthalpy (e.g., <code>`hOut`</code> for outdoor air enthalpy)

> **Sources: Set 1 Rules 3–4, Set 2 tables**

4. Control Signal Naming

****4.1**** Control ****input**** signals start with ``u``; control ****output**** signals

****4.1**** Control ****input**** signals start with ``u``; control ****output**** signals start with ``y``.

****4.2**** The ``u`/`y`` prefix may be ****omitted**** when the physical quantity already makes the direction clear (e.g., a temperature sensor output may simply be ``T`` or ``TOut`` rather than ``yT``).

****4.3**** When a controller has multiple outputs or inputs, augment with a descriptor (e.g., ``yVal`` and ``yDam`` for valve and damper output signals; ``uHea`` and ``uCoo`` for heating and cooling input signals).

> **Sources: Set 1 Rules 2 & 5, Set 2 conventions**

5. Prefix Conventions

****5.1**** The following prefixes are standard:

Prefix	Meaning	Examples
---	---	---
<code>`u`</code>	Control input signal	<code>`uOpeMod`</code> , <code>`uDam`</code> , <code>`uHea`</code> , <code>`uCoo`</code>
<code>`y`</code>	Control output signal	<code>`yHeaCoi`</code> , <code>`yCooCoi`</code> , <code>`yPosMin`</code> , <code>`yChiPumSpe`</code>
<code>`use_`</code>	Conditionally enabled input or feature (boolean parameter). <i>*"Should the model use the..."</i>	
<code>`have_`</code>	The system/zone **has** a particular component, sensor, or physical configuration (boolean parameter). <i>*"The system/zone has..."</i>	
<code>`need_`</code>	A required **control capability or operational constraint** (boolean parameter). <i>*"The system/zone needs..."</i>	

****5.2**** Names following ``use_``, ``have_``, and ``need_`` prefixes shall apply the same abbreviation conventions as other variables (see §2.2). Full words are acceptable only under the exceptions listed in §2.2.3.

> **Sources: Set 1 Rule 6, Set 2 tables**

6. Suffix / Postfix Conventions

6.1 Two Suffix Types

Suffixes fall into two categories, each with a distinct formatting rule:

Category	Formatting rule	Purpose	Standard set
---	---	---	---
Quantity-type markers	Underscore + full (unabbreviated) word	Classify the variable	
Value qualifiers	CamelCase, no underscore , following the three-character abbreviatio		

> **Rationale:** If the suffix is an unabbreviated word that classifies the variable's role, it takes an underscore. If it is an abbreviated qualifier that specifies the engineering context or constrains the value, it is written in CamelCase with no underscore. New underscore suffixes should be introduced sparingly and only for broadly applicable concepts used consistently across the library.

6.2 Standard Suffixes

Suffix	Type	Meaning	Examples
---	---	---	---
<code>`_flow`</code>	Quantity-type	Flow variable	<code>`Q_flow`</code> , <code>`m_flow`</code> , <code>`V_flow`</code> , <code>`VDis_flow`</code>
<code>`_nominal`</code>	Quantity-type	Design / nominal capacity	<code>`Q_flow_nominal`</code> , <code>`m_flow_nominal`</code>
<code>`_small`</code>	Quantity-type	Small value for regularisation	–
<code>`_remote`</code>	Quantity-type	From a remote sensor	<code>`dpChiWat_remote`</code>
<code>`Set`</code>	Value qualifier	Setpoint	<code>`TZonHeaSet`</code> , <code>`TZonCooSet`</code>
<code>`Min`</code>	Value qualifier	Minimum	<code>`TSupSetMin`</code> , <code>`lifMin`</code>
<code>`Max`</code>	Value qualifier	Maximum	<code>`TZonCooMax`</code> , <code>`VDisHeaSetMax_flow`</code>
<code>`Hea`</code>	Value qualifier	Heating qualifier	<code>`TZonHeaSet`</code> , <code>`zonDisEffHea`</code>
<code>`Coo`</code>	Value qualifier	Cooling qualifier	<code>`yCooCoi`</code> , <code>`zonDisEffCoo`</code>

6.3 Ordering of Multiple Suffixes

When a name requires multiple suffixes, follow this canonical order:

> **Quantity** → Context / Domain (``Sup``, ``Zon``, ``Dis``, ...) → Domain Qualifier
> (``Hea``, ``Coo``, ...) → ``Set`` → ``Min`` / ``Max`` → ``_flow`` → ``_nominal``

Example	Decomposition
---	---
<code>`TSupSetMin`</code>	<code>`T`</code> (quantity) → <code>`Sup`</code> (context) → <code>`Set`</code> → <code>`Min`</code>
<code>`TZonHeaSet`</code>	<code>`T`</code> (quantity) → <code>`Zon`</code> (context) → <code>`Hea`</code> (qualifier) → <code>`Set`</code>
<code>`VDisHeaSetMax_flow`</code>	<code>`V`</code> (quantity) → <code>`Dis`</code> (context) → <code>`Hea`</code> (qualifier) → <code>`Set`</code> → <code>`Max`</code> → <code>`_flow`</code>
<code>`m_flow_nominal`</code>	<code>`m`</code> (quantity) → <code>`_flow`</code> → <code>`_nominal`</code>

```
> **Important:** Value qualifiers such as `Min`, `Max`, `Set`, `Hea`, `Coo`  
> must always appear as **postfixes**, never as prefixes.  
> For example, use `chiLifMin`, **not** `minChiLif`.  
  
> *Sources: Set 1 Rule 6, Set 2 tables and examples*
```

7. Connector Naming

****7.1**** Paired connectors of the same domain (identical declarations, different icons) are distinguished by:

Convention	Usage	Example
---	---	---
`_a` / `_b`	Fluid ports	`port_a`, `port_b`
`_p` / `_n`	Electrical terminals	`terminal_p`, `terminal_n`

```
> *Source: Set 1 Rule 7*
```

8. Psychrometrics Package Naming (`Buildings.Utilities.Psychrometrics`)

****8.1**** Uppercase `X` = mass fraction per total mass; lowercase `x` = mass fraction per mass of dry air.

****8.2**** The notation `z\_xy` denotes a function or block with output `z` and inputs `x` and `y`.

****8.3**** Standard symbols:

Symbol	Meaning
---	---
`pW`	Water vapour pressure
`TDewPoi`	Dew point temperature
`TWetBul`	Wet bulb temperature
`TDryBul` (or `T`)	Dry bulb temperature

```
> *Source: Set 1 Rule 4*
```

Change Log (relative to previous consolidated draft)

#	Type	What changed	Section
---	---	---	---
C1	Contradiction resolved	Adopted `Min`/`Max` (camelCase, no underscore); removed `_min`/`_max`	2.1.1
C2	Contradiction resolved	Added explicit rule that `Min`, `Max`, `Set`, etc. must be **positive	2.1.1
A1	Ambiguity resolved	Formally defined `need_` prefix and distinguished it from `have_` and	2.1.1
A2	Ambiguity resolved	Defined two suffix categories (quantity-type markers vs. value qualifi	2.1.1
A3	Ambiguity resolved	Added guidance on when full (unabbreviated) words are acceptable §	2.1.1
A4	Ambiguity resolved	Standardised canonical postfix order with decomposition examples; co	2.1.1
A5	Ambiguity resolved	Clarified that "block instances" in §2 refers to instances, not clas	2.1.1
N1	New clarification	Changed "camelCase" → "UpperCamelCase (PascalCase)" for class names;	2.1.1
N2	New clarification	Separated `E` (general energy) and `Q` (thermal energy) into distinct	2.1.1
N3	New clarification	Replaced "approximately the first three characters" with "the first t	2.1.1
N4	New rule	Names after `use_`/`have_`/`need_` prefixes must follow §2.2 abbreviation con	2.1.1